Team 2 - Billy’s Amazing Team

Member Names: Mitchel Bekink, Alyx Bruno-Bamford, Ashley Bryan, Rajul Chawla, Bosco Lau, Kacey Van Der Walt, Chris Grzywacz

Continuous integration methods

We will utilise GitHub actions for this project to automate our continuous integration. It will allow us to complete many tasks such as unit testing on the JUnit framework, build testing using Gradle, automatic code formatting for consistency and creating artifacts to streamline debugging. All of this would be done automatically, in the background.

Unit testing is necessary to ensure that the components of this project are behaving as expected, creating a working game. Build testing ensures that our project can build into an executable program/file, it checks that any changes to the codebase do not break the build process. Code consistency ensures that our codebase follows formatting conventions, ensuring readability and uniformity. Artifacts (such as test reports) will allow us to view the test coverage/outputs of the unit tests, informing us for future development and debugging.

Continuous integration approaches

Our team intends to follow guidelines to allow for seamless development in the context of continuous integration, these guidelines include: modular testing, code reviews, maintaining a single repository and branching/merging often. These advisories will minimise problems occurring and will allow us to catch issues before they become too difficult to address.

Modular testing is the practice of breaking down our code into smaller components, making it easier to create unit tests for them. Code reviews ensure that, before merging into the main branch, the code being added is thoroughly assessed by other members. It also allows for merge conflicts to be identified and rectified, ensuring that nothing will break once merged. Branching and merging often is a good practice to follow because it will minimise merge conflicts, it also ensures that changes being made are up to date with the most recent version of the codebase.

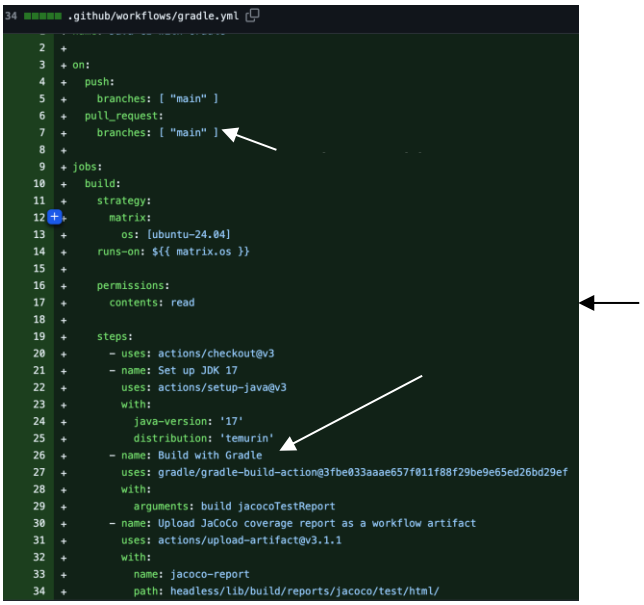
By keeping only one repository open that contains all of the code for our game, we can be sure that everyone has easy access to the only correct copy of our game. Our project is relatively small and simple. It would benefit most from a single, central repository that all members can acknowledge as holding the latest version of our codebase/assets.

Continuous integration pipeline

There is one pipeline for the project (due to the small scale) which manages all code changes to the main branch. It is triggered when there is a pull request or push to main. It takes in the repository files as input (java files, assets and gradle configuration). This pipeline will output a test report, this test report will indicate the coverage of the tests (how much code has been tested). If the tests fail, the changes will not be allowed into main. This ensures that all changes will maintain the integrity of the program under all circumstances.

Report on CI infrastructure for this project

In this project, we have utilised Continuous Integration infrastructure that is tailored to the specific needs of our product (compatibility, reliability and modularity). Our pipeline streamlines collaboration and automated testing, allowing for rapid feedback/improvement. It has helped us play a critical role in identifying and resolving issues during development. The pipeline is managed by Ash.



Our main continuous integration tool was Github actions under the Git version control. This is mainly because Github actions can seamlessly integrate into our version control on Github, making it easy to utilise and set up. Workflows (automated scripts that run tests) were triggered automatically when a member created a pull request, commit or branch merge. In addition, these scripts are written as *.yml files which are intuitive and flexible; simplifying the process for other members.

The file “grade.yml” is an example of automated testing where the codebase is copied to an external runner, alongside Java 17 and Gradle to generate an HTML report with JaCoCo (measuring and reporting code coverage).

Regarding tests, we utilised 2 main test types: unit and integration. Unit testing was used to test single modules within our codebase to ensure they output as expected under all circumstances. Integration testing is utilised to test that the code changes integrate into the existing system, ensuring no conflict. We utilised the JUnit testing framework which was particularly useful for unit testing. JUnit provides modules for assertion, setup and cleanup which is necessary for ensuring that specific modules are outputting as expected. Also, it allowed for all members to write unit tests, as they could be referred to within a workflow.

If there were an error (a build/test failing), the pipeline would not allow that merge request and the PR would fail. This ensures that all code changes do not break the project.

Our team established standards such as “branch often, merge often” to decrease the likelihood of PRs causing problems. We ensured a linear history to easily see the project’s commit history, it is also ideal for automated CI processes. The most important practice we follow is PR reviews, by ensuring two other members have reviewed the code before it is squashed and merged, we catch as many problems as possible beforehand. This saves us the time and energy required to revert to a previous state and re-do all the work.

To conclude, our CI infrastructure has an automated element (testing for successful building/outputs) and a human element (little and often changes, code reviewing) to ensure that our code is optimised and compatible.